

Lab 2: Quantum Algorithms

QML

1 introduction

in this lab we r implementing three foundational quantum algorithms: Deutsch-Jozsa, Bernstein-Vazirani, and Grover's search. these r the algorithms that first showed quantum computers can beat classical ones at certain tasks.

the point isn't just to run them — u need to understand the oracle pattern that shows up in all three, bcs it's the same pattern we'll see later in quantum machine learning.

by the end of this lab u should be able to:

- build oracle circuits for each algorithm
- implement and run all three algorithms in qiskit
- understand the query complexity advantage
- verify ur results against classical solutions

Key Point. all three algorithms follow the same template: prepare superposition $\rightarrow$ apply oracle $\rightarrow$ apply some transformation $\rightarrow$ measure. the magic is in how the oracle “kicks back” phase information into the superposition.

2 the oracle pattern

an oracle U_f implements a classical function $f: \{0,1\}^n \rightarrow \{0,1\}$ as a quantum gate. the standard way to do this is:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

but there's a trick. if u set the ancilla qubit to $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, then:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

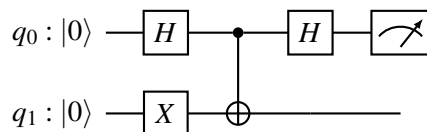
this is called phase kickback. the function value gets encoded as a phase on the input register, and the ancilla doesn't change. this is incredibly useful.

Exercise 1. phase kickback warmup. build a 2-qubit circuit where qubit 0 is the input and qubit 1 is the ancilla:

1. prepare qubit 1 in the $|-\rangle$ state (apply X then H)
2. apply H to qubit 0 to put it in superposition
3. apply CNOT with control 0 and target 1 (this implements $f(x) = x$)
4. apply H to qubit 0

5. measure qubit 0

the circuit looks like this:



what do you measure on qubit 0? run it 1024 times. explain why you get this result using phase kickback.

3 Deutsch-Jozsa algorithm

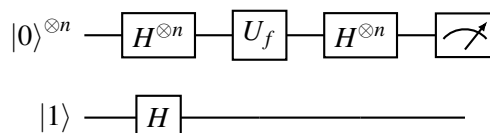
3.1 the problem

you have a black-box function $f: \{0,1\}^n \rightarrow \{0,1\}$. you are promised that f is either:

- constant: $f(x) = 0$ for all x , or $f(x) = 1$ for all x
- balanced: $f(x) = 0$ for exactly half the inputs, $f(x) = 1$ for the other half

classically you need up to $2^{n-1} + 1$ queries to decide. quantum: 1 query. that's an exponential speedup.

3.2 the algorithm

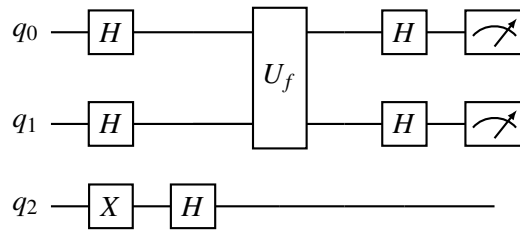


steps:

1. start with n qubits in $|0\rangle$ and 1 ancilla in $|1\rangle$
2. apply H to everything
3. apply the oracle U_f
4. apply H to the n input qubits
5. measure the input qubits

if you measure all zeros $\rightarrow f$ is constant. anything else $\rightarrow f$ is balanced.

here's the 2-qubit version more explicitly:



Exercise 1. Deutsch-Jozsa oracles. build oracle circuits for the following functions on 2 input qubits (plus 1 ancilla):

1. $f(x) = 0$ for all x (constant) — this one's easy, it's literally the identity
2. $f(x) = 1$ for all x (constant) — just flip the ancilla
3. $f(x_0, x_1) = x_0 \oplus x_1$ (balanced) — think about wt gate does XOR
4. $f(x_0, x_1) = x_0$ (balanced) — even simpler

for each oracle: wrap it in the full Deutsch-Jozsa algorithm (H's, oracle, H's, measure). run it and verify u get the correct answer (all-zeros for constant, not-all-zeros for balanced).

Exercise 2. scaling Deutsch-Jozsa. write a function that creates a random balanced oracle on n qubits. here's one way to do it:

- randomly pick a bit string $s \in \{0, 1\}^n$ (not all zeros)
- the oracle computes $f(x) = s \cdot x \pmod 2$ (inner product mod 2)
- implement this using CNOT gates: for each bit i where $s_i = 1$, apply CNOT from qubit i to the ancilla

test ur Deutsch-Jozsa implementation for $n = 3, 4, 5, 6$. does it always correctly identify balanced vs constant?

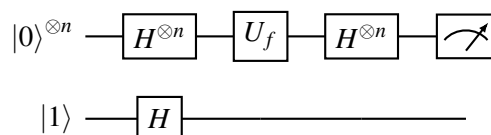
4 Bernstein-Vazirani algorithm

4.1 the problem

u have an oracle that computes $f(x) = s \cdot x \pmod 2$ for some hidden bit string s . u want to find s .
classically: u need n queries (try each standard basis vector). quantumly: 1 query.

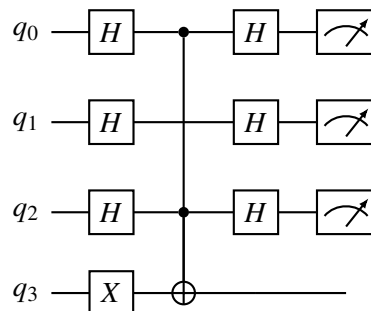
4.2 the algorithm

here's the beautiful thing: the circuit is exactly the same as Deutsch-Jozsa. the only difference is the oracle and how u interpret the measurement result.



when u measure the input qubits, u directly get the hidden string s . that's it. one shot.

for example, if $s = 101$ and we have 3 input qubits:



notice the oracle: CNOT from qubit 0 to ancilla (bcs $s_0 = 1$), nothing for qubit 1 (bcs $s_1 = 0$), CNOT from qubit 2 to ancilla (bcs $s_2 = 1$).

Exercise 1. Bernstein-Vazirani.

1. implement the oracle for $s = 1011$ (4 input qubits + 1 ancilla). the oracle should apply CNOT from qubit i to the ancilla whenever $s_i = 1$.
2. wrap it in the full BV algorithm and run it. verify u recover $s = 1011$.
3. write a function `bv_oracle(s)` that takes a bit string and returns a qiskit circuit implementing the oracle.
4. write a function `run_bv(s)` that runs the full algorithm and returns the recovered string.
5. test with at least 5 different hidden strings of varying lengths.
6. how many shots do u need? theoretically u should only need 1 shot. try it with 1 shot vs 100 shots. does it always work with 1 shot on the simulator?

5 Grover's search algorithm

5.1 the problem

u have an unstructured database of $N = 2^n$ items, and u want to find the one item that satisfies some condition (marked by an oracle). classically: $O(N)$ queries. Grover's: $O(\sqrt{N})$ queries.

this isn't an exponential speedup like Deutsch-Jozsa, but it's provably optimal for unstructured search, which is pretty sick.

5.2 the algorithm

Grover's has two main pieces that u repeat:

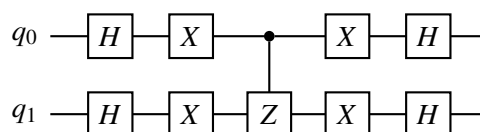
1. oracle U_ω : flips the phase of the target state. $U_\omega |x\rangle = (-1)^{f(x)} |x\rangle$
2. diffuser U_s : reflects about the uniform superposition. $U_s = 2|s\rangle\langle s| - I$ where $|s\rangle = H^{\otimes n} |0\rangle^{\otimes n}$

the full algorithm:

1. apply $H^{\otimes n}$ to create uniform superposition
2. repeat $\approx \frac{\pi}{4}\sqrt{N}$ times: apply oracle, then apply diffuser
3. measure

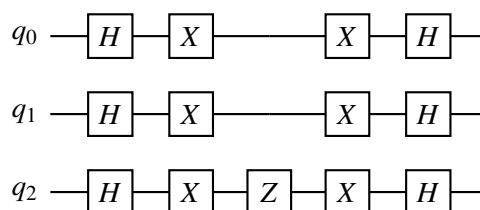
5.3 the diffuser circuit

the diffuser for 2 qubits looks like this:



for more qubits, the middle part becomes a multi-controlled Z gate. u can implement this using a multi-controlled X (Toffoli-like) sandwiched between H gates on the target.

for 3 qubits the diffuser is:



Key Point. the diffuser is the same regardless of wt u're searching for. only the oracle changes.

Exercise 1. 2-qubit Grover's. let's search for the state $|11\rangle$ among 4 possible states.

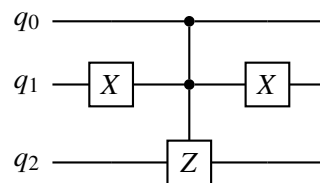
1. build the oracle for $|11\rangle$: this should flip the phase of $|11\rangle$ only. u can do this with a controlled-Z gate (in qiskit: `.cz(0,1)`).
2. build the diffuser: H on both, X on both, CZ, X on both, H on both.
3. put it all together: H on both qubits, then oracle, then diffuser, then measure.
4. run it. u should get $|11\rangle$ with probability 1 (or close to it).
5. how many Grover iterations did u use? how many does the formula $\frac{\pi}{4}\sqrt{N}$ predict?

Exercise 2. 3-qubit Grover's. now search for a target state in a space of $N = 8$.

1. build an oracle for the target state $|101\rangle$. hint: use X gates to flip the qubits that should be $|0\rangle$, apply a multi-controlled Z, then unflip.
2. build the 3-qubit diffuser.
3. the formula says u need $\approx \frac{\pi}{4}\sqrt{8} \approx 2.2$ iterations, so try 2 iterations.

- run with 2048 shots. wt probability do u measure $|101\rangle$?
- try with 1, 2, and 3 iterations. which gives the highest probability? plot the success probability vs number of iterations.

here's wt the oracle for $|101\rangle$ looks like (the X gates flip qubits that should be 0):



(this is a simplified version — the multi-controlled Z in the middle marks the $|101\rangle$ state)

Exercise 3. general Grover's implementation. write a reusable Grover's implementation:

- write a function `grover_oracle(target, n)` that creates an oracle circuit marking the target bitstring
- write a function `diffuser(n)` that creates the n -qubit diffuser
- write a function `grovers(target, n, num_iterations)` that assembles the full algorithm
- test with various targets on 2, 3, and 4 qubits
- for each n , find the optimal number of iterations experimentally and compare with $\frac{\pi}{4}\sqrt{2^n}$

Exercise 4. Grover's with multiple solutions. wt happens when there r multiple marked items?

- modify ur oracle to mark 2 target states (e.g., both $|101\rangle$ and $|110\rangle$)
- run Grover's with varying iterations
- the optimal number of iterations when there r k solutions out of N is $\frac{\pi}{4}\sqrt{N/k}$. verify this experimentally.
- wt goes wrong if u use too many iterations?

6 comparison and reflection

Exercise 1. wrap-up. fill in this table:

algorithm	classical queries	quantum queries	speedup
Deutsch-Jozsa	$2^{n-1} + 1$	1	?
Bernstein-Vazirani	n	1	?
Grover's	$O(N)$	$O(\sqrt{N})$	?

answer these questions:

1. which algorithm gives the biggest speedup? is it a “useful” speedup or more of a theoretical curiosity?
2. what’s the common pattern across all three algorithms? (think about the role of H gates and phase kickback)
3. how does Grover’s differ fundamentally from the other two? (hint: it’s iterative)
4. which of these algorithms is most relevant to machine learning? why? (we’ll revisit this in lab 5)

7 submission

submit a jupyter notebook with:

- all exercises completed with code and output
- circuit diagrams for each algorithm
- the comparison table filled in
- a brief paragraph reflecting on the oracle pattern and how it might connect to ML